FAULTLINE · T1.6

The Quarantine Rule

You are writing the exam and sitting it. This is the one rule that stops you from cheating without noticing — and the reason it has to be decided today, not later.

The problem, plainly

Faultline grades itself. You break the system on purpose, so you already know the answer before the agent starts looking. That is what makes the benchmark possible — and it is also the trap.

Over the next few months you will tune this thing constantly. Change a prompt, the score goes up, keep the change. Adjust what evidence gets fed in, score goes up, keep it. That loop is how the project improves. It is also how a benchmark quietly stops measuring anything.

> If you tune against the same scenarios you grade on, your final number measures how well you fitted the answers — not how well the system diagnoses.

Every self-graded benchmark has this problem. Most projects never mention it. Saying it out loud, and building the guard rails in code, is a large part of what makes Faultline credible rather than a demo.

Two ways to cheat — and they are not the same

This is the bit worth actually understanding, because the fix for one does nothing about the other.

AXIS 1 — TUNING ON THE TEST

You tweak something, the score improves, you keep the tweak. Repeat two hundred times. Every scenario you did that against has now been fitted to, and can no longer grade you honestly.

The fix: lock away three of the ten scenarios. Never tune against them. Never even look at them while debugging. They are the only honest grade you have.

AXIS 2 — READING THE ANSWER KEY

When you rehearse a scenario you write down what really broke. Those notes later become the agent's memory of past incidents. So when the agent investigates that same scenario, the closest match in its memory is your own answer, in your own words. It doesn't diagnose. It looks up.

The fix: tag every memory with where it came from, and hide a scenario's own notes while that scenario is being graded. Built later, at T4.1b.

Here is the part people miss. Axis 1's fix is blind to Axis 2. A dev-set scenario looking up its own dev-set rehearsal breaks no split rule whatsoever — both sides are dev. The quarantine is intact and the score is still meaningless. Two problems, two mechanisms, two different phases of the project.

Why this had to happen before writing scenarios

The instant you rehearse a scenario, you create files — logs, metrics, notes on what broke. Those files feed the agent's memory later.

If the split doesn't exist yet, holdout material is already sitting in the pile, mixed in with everything else. You cannot unmix it afterwards without redoing every rehearsal from scratch.

So the split has to be decided while there is still nothing to contaminate. That is the whole reason T1.6 sits *before* T1.5 in the plan, even though deciding a split before you know what you're splitting feels backwards.

What was actually decided

Ten scenarios, seven for development, three held out. Roughly 70/30, spread across your four fault classes.

The unusual part: the splits were assigned to **empty numbered slots**, before any scenario had a name or a description. You now fill slots in order — the first bad-deploy scenario you write becomes `bad_deploy-1` and inherits whatever that slot says.

That is deliberate. If you assign splits after writing the scenarios, you will put the clean, well-behaved one in the holdout and the awkward one in dev — not dishonestly, just because the awkward one feels like it needs work. Splitting blind removes the choice entirely.

SLOT	FAULT CLASS	SPLIT
bad_deploy-1	Bad deploy	DEV
bad_deploy-2	Bad deploy	HOLDOUT
bad_deploy-3	Bad deploy	DEV
dependency_latency-1	Dependency latency	DEV
dependency_latency-2	Dependency latency	HOLDOUT
resource_exhaustion-1	Resource exhaustion	DEV
resource_exhaustion-2	Resource exhaustion	DEV
resource_exhaustion-3	Resource exhaustion	HOLDOUT
bad_config-1	Bad config	DEV
bad_config-2	Bad config	DEV

One honest gap

Three holdout slots cannot cover four fault classes. Bad config is dev-only, so nothing tests whether the system generalises to an unseen config fault.

The reasoning: bad config and bad deploy are diagnosed the same way — spot that something changed just before things broke, then undo the change. A system that handles a held-out bad deploy will very likely handle a bad config too. So it is the cheapest coverage to skip.

It is still a real gap. Say so when you report numbers. T7.1 grows the catalog to thirty-plus and every class gets holdout coverage then.

The four files this added

`docs/adr/0008-contamination-model.md`	The decision record. Both axes, why the first is blind to the second, who enforces which, and the policy on which number gets published. This is the file an interviewer would find.
`evals/scenarios/SPLIT.md`	The slot table above, in writing, committed before authoring. Marked do-not-edit, because editing it to accommodate a scenario defeats the entire point.
`tests/test_contamination.py`	Seven tests that fail the build if a rule is broken — artifacts filed on the wrong side, more scenarios than slots, or the doc drifting from the code.
`evals/scenarios/artifacts/dev/` `evals/scenarios/artifacts/holdout/`	Two empty folders, created now so that rehearsal output has nowhere unlabelled to land. Quarantine becomes a property of the file path rather than something you have to remember.

Who enforces what, and when

The rules land in stages. Only the first two are your problem right now.

PHASE	ENFORCES	HOW
T1.6 · now	Split exists before rehearsals	Written into each scenario's YAML at authoring; folders reject unlabelled artifacts
T1.5 · next	You fill slots in order	Manual discipline, checked by the test suite
T2.4b	Only dev material enters memory	Knowledge stores seeded from the dev split only, each record stamped with its origin
T4.1	Holdout stays sealed during scoring	Harness refuses to score a holdout scenario whose files appear in any corpus
T4.1b	Axis 2 — no reading your own answer	Retrieval filters out the scenario under test; a run where the filter didn't fire is marked invalid, not just noted
T7.1	Closes the bad-config gap	Catalog grows to 30+, holdout reaches 9–10, headline switches to holdout-only

Rules from here on

- ✓ Assign a scenario's split when you create the file, before you rehearse it even once.
- ✓ Put rehearsal output in `artifacts/dev/<id>/` or `artifacts/holdout/<id>/`. Nowhere else.
- ✓ When you publish a score, state how many scenarios it covers and show the dev/holdout breakdown.
- × Never tune a prompt, a retrieval setting, or the corpus against a holdout scenario. Not even once, not even to debug.
- × Never edit `SPLIT.md` because a scenario would be more convenient on the other side.
- × Never headline a holdout-only number while the holdout is three scenarios. Three is an anecdote.

The sixty-second version, for interviews

You will get asked how you know your accuracy number is real. This is the answer.

> “It's a self-labelled benchmark, so contamination is the first thing to worry about. There are two separate leaks and I handle them differently.
>
> The first is tuning on the test set — so I declared a 70/30 dev/holdout split *before* authoring any scenarios, assigned to unnamed slots so I couldn't route the easy ones to the holdout. Nothing tuned ever touches holdout.
>
> The second is subtler. Scenario rehearsals seed the past-incident store, so when the agent investigates a scenario, the nearest neighbour in retrieval is that scenario's own rehearsal — containing the true root cause, in my words. The split doesn't catch that, because it's within-split self-reference. So every knowledge artifact carries a provenance tag, and the harness excludes the scenario under test from retrieval and asserts the filter actually fired. A run where it didn't fire is marked invalid, not just flagged.
>
> And with a three-scenario holdout I don't headline holdout-only numbers — I report the full set with the split labelled and n stated, until the catalog reaches thirty.”

That answer is worth more than most of the code. It shows you know how measurement goes wrong before anyone tells you.

Glossary

`dev split`

The seven scenarios you're allowed to work against — debug with, tune against, look at freely.

`holdout`

The three you sealed. Untouched by any tuning, so their score is the only unbiased estimate of how the system performs on something it hasn't been fitted to.

`contamination`

Any path by which knowledge of the answer reaches the system being graded. Two paths here: tuning against a scenario, and retrieving that scenario's own write-up.

`provenance`

A stamp on every piece of the agent's knowledge saying where it came from — hand-written by you, or generated from scenario X. Without it you can't filter anything out later.

`self-exclusion`

Hiding a scenario's own artifacts from the agent while that scenario is being graded. The Axis 2 fix, arriving at T4.1b.

`n`

How many scenarios a number is based on. An accuracy figure without n attached is not a measurement.

`headline number`

The accuracy figure that goes at the top of your README. Full-set with the split labelled for now; holdout-only once the holdout is big enough to mean something.
